

MQTT Communication Experiment

MQTT Communication Experiment

1. MQTT Overview
2. Core Concepts
 - 2.1 Publish/Subscribe Model
 - 2.2 Configuration Parameters
 - 2.3 Core API
 - 2.4 Topics and Messages
3. Hardware Dependencies
4. Project Architecture and Flow
5. Source Code Explained
 - 5.1 Creating the MQTT Client
 - 5.2 Message Callback
 - 5.3 Main Loop
6. Operating Procedures
 - 6.1 Flashing and Running
 - 6.2 Serial Output Example
 - 6.3 Verification Methods
7. Common Problems

1. MQTT Overview

MQTT (Message Queuing Telemetry Transport) is the most mainstream lightweight publish/subscribe messaging protocol in the IoT field. It is based on TCP and uses the **publish/subscribe (Pub/Sub)** mode instead of the traditional client-server direct connection mode, achieving **temporal and spatial decoupling between producers and consumers**.

In this experiment the ESP32-S3 connects to the public Broker `broker.emqx.io`, publishes messages to the `test/esp32/send` topic, and subscribes to the `test/esp32/recv` topic.

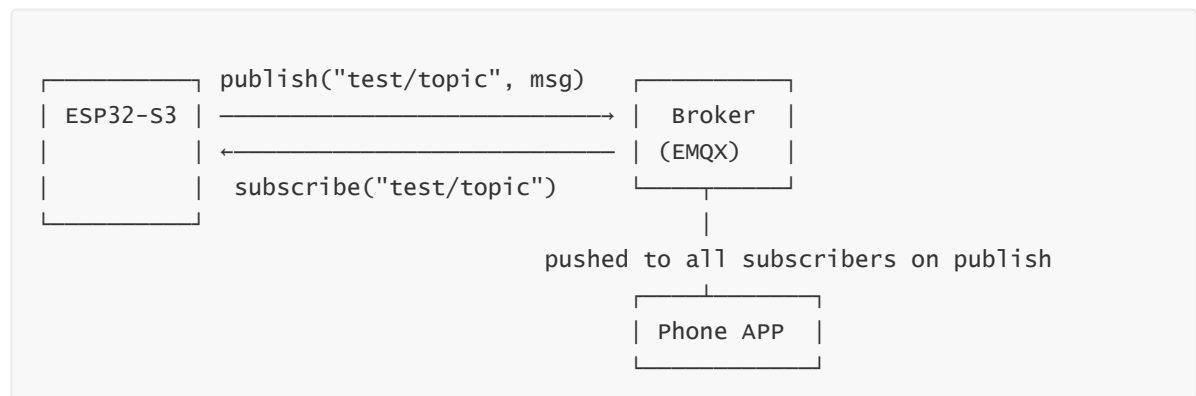
Typical application scenarios:

Application Type	Example
Sensor reporting	Temperature/humidity published to the cloud periodically
Remote control	Phone App publishes control commands, ESP32 subscribes and receives
Multi-device linkage	Door magnet sensor triggers -> alarm responds
Smart home	Manage devices through platforms such as Home Assistant / EMQX

This experiment depends on the `umqtt.simple` library (built into MicroPython or uploaded separately). Using a public Broker requires no self-built server.

2. Core Concepts

2.1 Publish/Subscribe Model



- **Broker:** the message relay station, responsible for receiving publishes and pushing to subscribers
- **Topic:** the message topic, e.g. `home/livingroom/temperature`
- **Publish:** publish a message to a specified topic
- **Subscribe:** subscribe to a topic; the Broker pushes messages on that topic to you

2.2 Configuration Parameters

Parameter	Description	Value Used in This Experiment
BROKER	MQTT broker address	<code>broker.emqx.io</code> (public)
TOPIC_PUB	Publish topic	<code>test/esp32/send</code>
TOPIC_SUB	Subscribe topic	<code>test/esp32/recv</code>
CLIENT_ID	Client identifier (must be unique)	<code>esp32s3_mqtt</code>

2.3 Core API

```
from umqtt.simple import MQTTClient

# create
client = MQTTClient("client_id", "broker.emqx.io", port=1883)
client.set_callback(on_message)      # set callback
client.connect()                     # connect Broker
client.subscribe("topic/sub")        # subscribe
client.publish("topic/pub", "hello") # publish
client.check_msg()                   # check for subscribed messages
```

2.4 Topics and Messages

This experiment defines two topics; the ESP32 uses them to interact with external clients (such as MQTTX):

Topic	Direction	QoS	Message Content	Period
<code>test/esp32/send</code>	ESP32 -> Broker	0	"Hello ESP32 MQTT"	Every 3s
<code>test/esp32/recv</code>	Broker -> ESP32	0	Any text (sent by MQTTX)	Determined by the other side

QoS 0 (At most once): the message is delivered at most once and delivery is not guaranteed. Suitable for lightweight periodic sensor reporting.

Message flow:

```
ESP32-S3 --publish("test/esp32/send", "Hello ESP32 MQTT")--> Broker --forward-
-> MQTTX (subscriber)
ESP32-S3 <--subscribe("test/esp32/recv")----- Broker <--publish--
MQTTX (publisher)
```

The ESP32 publishes and subscribes to **separate topics** to avoid receiving its own messages. If you want a self-send/self-receive loopback test, set `TOPIC_PUB` and `TOPIC_SUB` to the same topic.

3. Hardware Dependencies

The ESP32-S3 has built-in WiFi and needs access to the public Broker (`broker.emqx.io`). No PC or self-built server is required.

4. Project Architecture and Flow

```
Script execution (single-file .py)
|
|- wifi_init(): connect to WiFi (STA mode)
|- create MQTTClient -> connect() -> subscribe()
|
|- while True:
|   |- publish(TOPIC_PUB, "Hello ESP32 MQTT") // publish
|   |- check_msg() // check subscription
|   `- sleep(3s)
```

5. Source Code Explained

Full source code: `Source Code -> MicroPython -> 4.Network Protocols -> MQTT -> MQTT.py`

5.1 Creating the MQTT Client

Use the `umqtt.simple` library to connect to the public Broker `broker.emqx.io:1883`, set the callback, and subscribe to the target topic.

```

from umqtt.simple import MQTTClient

BROKER      = "broker.emqx.io"          # public broker
TOPIC_PUB   = "test/esp32/send"         # publish topic
TOPIC_SUB   = "test/esp32/recv"         # subscribe topic
CLIENT_ID   = "esp32s3_mqtt"           # unique client ID

client = MQTTClient(CLIENT_ID, BROKER, port=1883)
client.set_callback(on_message)         # callback on message
client.connect()
client.subscribe(TOPIC_SUB)

```

5.2 Message Callback

Automatically called when the Broker pushes a subscribed message; the parameters are topic and msg (both bytes).

```

def on_message(topic, msg):
    """received subscribed message"""
    print(f"[Received] {topic.decode()} -> {msg.decode()}")

```

5.3 Main Loop

Publishes one message every 3s -> `check_msg()` checks whether any subscribed message has arrived.

```

while True:
    msg = "Hello ESP32 MQTT"
    client.publish(TOPIC_PUB, msg)      # publish
    print(f"[Sent] {msg}")

    client.check_msg()                  # check for messages
    time.sleep(3)

```

6. Operating Procedures

6.1 Flashing and Running

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3. Thonny IDE`.

1. Modify `WIFI_SSID` / `WIFI_PASS`

```

WIFI_SSID = "Yahboom2"          # WiFi名称 / WiFi SSID
WIFI_PASS = "yahboom890729"     # WiFi密码 / WiFi password

BROKER     = "broker.emqx.io"    # MQTT Broker 地址
TOPIC_PUB  = "test/esp32/send"   # 发布主题 / publish topic
TOPIC_SUB  = "test/esp32/recv"   # 订阅主题 / subscribe topic
CLIENT_ID  = "esp32s3_mqtt"      # 客户端 ID (需唯一) / unique client ID

```

2. Run `MQTT.py`



-
- The screenshot shows the MQTTX application interface. The left sidebar contains navigation icons. The main panel is divided into two sections. The top section, titled 'Connections', shows a list of subscriptions. A red box highlights the subscription 'test/esp32/send' with QoS 0, and a red arrow points to it. Below this, a message history shows several 'Hello ESP32 MQTT' messages. A red box highlights the 'Topic: test/esp32/send' and 'QoS: 0' fields. At the bottom, the 'test/esp32/recv' subscription is highlighted with a red box, and the message payload is shown as 'msg: 'hello''.

Plaintext
All
Received
Published

Hello ESP32 MQTT
2026-08-07 10:03:53:781

Topic: test/esp32/recv
QoS: 0
"msg": "hello"
2026-08-07 10:03:56:820

Topic: test/esp32/recv
QoS: 0
"msg": "hello"
2026-08-07 10:03:57:073

Topic: test/esp32/send
QoS: 0
Hello ESP32 MQTT
2026-08-07 10:03:57:371

Plaintext
QoS 0
☐ Retain
Meta
2/2

test/esp32/recv
"msg": "hello"

6.2 Serial Output Example

```

===== MQTT CLIENT =====
[WiFi] Connecting to 'Yahboom2' ...
[WiFi] Connected! IP: 192.168.11.229
[MQTT] Connected to broker.emqx.io
[MQTT] Subscribed: test/esp32/recv
[Sent] Hello ESP32 MQTT
[Sent] Hello ESP32 MQTT
[Recv] test/esp32/recv -> "msg": "hello"

[Recv] test/esp32/recv -> "msg": "hello"

[Recv] test/esp32/recv -> "msg": "hello"

[Sent] Hello ESP32 MQTT

```

```
[Sent] Hello ESP32 MQTT
```

```
===== MQTT CLIENT =====  
[WiFi] Connecting to 'Yahboom2' ...  
[WiFi] Connected! IP: 192.168.11.229  
[MQTT] Connected to broker.emqx.io  
[MQTT] Subscribed: test/esp32/recv  
[Sent] Hello ESP32 MQTT  
[Sent] Hello ESP32 MQTT  
[Recv] test/esp32/recv -> "msg": "hello"  
  
[Recv] test/esp32/recv -> "msg": "hello"  
  
[Recv] test/esp32/recv -> "msg": "hello"  
  
[Sent] Hello ESP32 MQTT  
[Sent] Hello ESP32 MQTT
```

6.3 Verification Methods

Operation	Expected Result
Subscribe to <code>test/esp32/send</code> with MQTTX	Receives "Hello ESP32 MQTT" every 3s
Use MQTTX to send <code>hello</code> to <code>test/esp32/recv</code>	The ESP32 serial prints <code>[Received]</code> <code>test/esp32/recv -> hello</code>

7. Common Problems

Symptom	Cause	Solution
<code>ImportError: no module named 'umqtt'</code>	Firmware does not include the umqtt library	Upload <code>umqtt/simple.py</code> to the ESP32 manually
Timeout connecting to Broker	Network unreachable or Broker unreachable	<code>ping broker.emqx.io</code> to test reachability
<code>check_msg()</code> has no callback	<code>set_callback()</code> or <code>subscribe()</code> not called	Confirm all three steps in section 5.1 are complete
Client ID conflict	Two devices use the same ID	Use a unique ID, e.g. <code>esp32s3_mqtt_<last 4 of MAC></code>
No auto-reconnect after Broker disconnect	<code>umqtt.simple</code> has no built-in reconnect mechanism	Detect <code>OSError</code> in <code>try/except</code> ; after disconnect, call <code>connect()</code> + <code>subscribe()</code> again

